企业级智能知识库问答系统 PRD

项目名称：omni-knowledge（知全）

版本：v1.0 Final
日期：2026-02-26
负责人：wangjifei
技术栈：LlamaIndex Agents + 私有化部署

一、产品定位

企业级RAG系统，核心能力：

1. **全格式文档解析**（PDF/TXT/MDWord/Excel/图片/语音）
2. **跨文档精准问答**（语义检索 + 推理）
3. **句子级引用溯源**（原文高亮定位）
4. **私有化部署**（数据不出内网）

核心价值：检索时间 30分钟→秒级，答案可信度提升90%+

二、核心挑战（需重点攻克）

⚠️ 问题1：元数据统计类查询失效

- **场景**："有多少份文档？"、"安全部的制度有哪些？"
- **失效原因**：向量检索基于内容语义，无法处理结构化统计
- **关键点**：需实现意图分类 + Text-to-SQL

⚠️ 问题2：高相似文档区分困难

- **场景**：《火灾预案》《水灾预案》结构完全一致，正文省略主语
- **失效原因**：Embedding相似度过高，文档名称权重不足
- **关键点**：需强化元数据过滤 + 实体识别预处理

⚠️ 问题3：文档分块破坏语义

- **场景**：流程被切断、表格被拆散、列表不完整
- **失效原因**：固定长度分块忽略语义边界
- **关键点**：需语义感知分块 + 动态上下文扩展

⚠️ 问题4：表格内容处理失真

- **场景**：合并单元格丢失、列对应关系错乱、数字识别错误

- **失效原因**：PDF表格解析技术难度高
- **关键点**：需高精度解析工具 + 表格专用Prompt

三、系统架构

3.1 技术栈

层级	核心组件	选型
AI核心	RAG框架	LlamaIndex 0.10+
	LLM	国内主流开源大模型（符合openai风格接口）
	Embedding	标准Embedding规范的API即成
	Reranker	bge-reranker-large
文档解析	PDF	LlamaParse（表格识别最佳）
	Word/Excel	Unstructured.io + pandas
	OCR	PaddleOCR
存储	向量库	Milvus 2.3+
	关系库	PostgreSQL 15+
	对象存储	MinIO
后端	框架	FastAPI + Celery
前端	框架	React + Ant Design + PDF.js

3.2 架构分层

```
用户层：Web界面 / API接口
  ↓
应用层：文档管理 / 智能问答 / 引用溯源 / 权限管理
  ↓
AI层：LlamaIndex Agents
  ├─ Query Engine（混合检索 + Rerank）
  ├─ Agent Orchestrator（意图分类 + 任务编排）
  └─ Citation Tracker（引用追踪）
  ↓
解析层：文档解析 + 智能分块 + Embedding
  ↓
存储层：Milvus / PostgreSQL / MinIO / Redis
  ↓
模型层：LLM服务 / Embedding服务 / Reranker服务
```

3.3 部署方案（私有化）

硬件配置：

- GPU推理节点：2台 × 8×A100 80GB（LLM + Embedding）
- 应用节点：3台 × 32核128G（业务服务）
- 存储节点：3台 × 16核64G + 20TB SSD（数据库集群）

容器化：Docker + K8s，GPU节点使用vLLM推理加速

四、核心功能设计

4.1 文档管理

上传支持：

- 格式：PDF/Word/Excel/PPT/图片/音视频/网页
- 方式：单文件/批量/URL抓取/API对接

元数据字段（必填：文档名、部门、类别、密级）：

```
{
  "doc_name": "火灾应急预案",
  "department": "安全部",
  "category": "应急预案",
  "tags": ["火灾", "应急"],
  "security_level": "内部",
  "version": "v2.0",
  "effective_date": "2024-01-01"
}
```

解析流程：

```
上传 → 格式识别 → 内容提取（文本/表格/图片）
     → 质量检测（乱码/空白页/重复）
     → 智能分块（保留章节边界、表格完整性）
     → 向量化（文本+图片Embedding）
     → 入库（Milvus + PostgreSQL + MinIO）
```

功能要求：

- 支持版本管理（历史版本、版本比对）
- 支持权限控制（文档级 + 部门级）
- 支持统计分析（访问热度、存储占用）

4.2 智能问答

交互界面要求：

- 搜索框支持高级选项（文档范围、搜索模式、答案详细度）
- 答案区展示引用列表（文档名/章节/页码/原文片段）
- 支持追问、不满意反馈、导出

问答流程：

```
用户提问
  ↓
预处理：查询改写 + 实体识别 + 意图分类
  ↓
路由决策：
  ├─ 元数据查询 → Text-to-SQL
  ├─ 内容查询   → RAG检索
  └─ 混合查询   → 两者结合
  ↓
RAG检索：
  混合检索（向量Top50 + BM25 Top30 + 元数据过滤）
  → Rerank重排序
  → 上下文扩展（前后chunk + 同章节）
  → LLM生成（含引用要求）
  ↓
后处理：质量检测 + 敏感信息脱敏 + 置信度计算
  ↓
返回：答案 + 引用 + 置信度 + 相关推荐
```

LlamaIndex Agents 设计思路：

- 定义工具集：`vector_search_tool`（内容检索）、`sql_query_tool`（元数据统计）
- Agent自动推理：识别问题类型 → 选择工具 → 组合结果
- 示例：问"安全部本月新增的制度讲了什么？" → 先SQL查ID列表 → 再向量检索内容

4.3 引用溯源

引用信息结构：

```
{
  "doc_name": "火灾应急预案",
  "chapter": "第3章",
  "section": "3.2节",
  "page": 5,
  "original_text": "发现火情后立即拨打119...",
  "confidence": 0.92
}
```

原文定位方案：

- PDF高亮：使用PDF.js定位坐标，支持句子级高亮
- 存储映射：chunk_id → (doc_id, page, position)

- 交互：点击引用 → 弹窗显示原文 → 支持跳转PDF查看

五、关键技术方案

5.1 智能分块策略

目标：保留语义完整性，避免切断流程/表格/列表

方案要点：

- 按章节边界优先切分（识别标题层级）
- 表格作为独立chunk，不拆分
- 长表格：整体Embedding + 按行切分（保留表头）
- 每个chunk注入元数据：`{doc_name, chapter, page, chunk_type}`

5.2 混合检索策略

组合方式：

1. 向量检索：`query_embedding` 在 Milvus 中检索 Top 50
2. 关键词检索：BM25 算法在 PostgreSQL 全文索引中检索 Top 30
3. 元数据过滤：根据用户选择的部门/类别/时间过滤
4. 融合排序：RRF（Reciprocal Rank Fusion）算法合并
5. Rerank：bge-reranker 重新打分，取 Top 10

5.3 元数据查询实现

Text-to-SQL流程：

```
用户问题："有多少份应急预案？"
  ↓
意图识别：元数据统计类
  ↓
LLM生成SQL：
  SELECT COUNT(*) FROM documents
  WHERE category = '应急预案'
  ↓
SQL注入检测 → 执行查询 → 返回："共有12份应急预案"
```

数据库表设计（核心字段）：

```sql
-- 文档表
CREATE TABLE documents (
  doc_id VARCHAR PRIMARY KEY,
  doc_name VARCHAR,
  department VARCHAR,
  category VARCHAR,
```

```sql
  security_level VARCHAR,
  upload_time TIMESTAMP,
  version VARCHAR
);

-- 章节表
CREATE TABLE chapters (
  chapter_id VARCHAR PRIMARY KEY,
  doc_id VARCHAR,
  chapter_title VARCHAR,
  level INT,
  parent_id VARCHAR
);

-- 标签表
CREATE TABLE tags (
  doc_id VARCHAR,
  tag_name VARCHAR
);
```

5.4 表格处理方案

解析工具选择：

- PDF表格：LlamaParse（商业版，识别率最高）
- Excel：pandas直接读取
- Word表格：python-docx

表格保存格式：

```json
{
  "type": "table",
  "headers": ["部门", "负责人", "电话"],
  "rows": [
    ["安全部", "张三", "138xxxx"],
    ["安全部", "李四（副）", "139xxxx"]
  ],
  "markdown": "| 部门 | 负责人 | 电话 |\n|------|--------|------|\n..."
}
```

LLM Prompt优化：

```
以下是表格内容（Markdown格式）：
| 部门 | 负责人 | 电话 |
|------|--------|------|
| 安全部 | 张三 | 138xxxx |

请注意：
1. 表格每行代表一条记录
2. 相同单元格表示合并关系
3. 严格按照表格内容回答，不要推测
```

六、性能与安全

6.1 性能指标

指标	目标值	说明
问答响应时间	< 3秒	P95
文档解析速度	100页/分钟	PDF平均
并发支持	500 QPS	峰值
检索准确率	> 90%	Top 5命中
表格识别准确率	> 95%	人工抽检

6.2 安全策略

- **数据安全**：全链路加密（传输TLS + 存储AES-256）
- **权限管控**：基于角色的访问控制（RBAC）
- **审计日志**：记录所有查询和下载操作
- **敏感信息脱敏**：自动识别并脱敏手机号/身份证
- **水印保护**：导出PDF自动添加用户水印

七、智能问答产品界面详细设计

7.1 整体布局

```
┌──────────────────────────────────────────────────────────┐
│  [Logo] 智能知识库                    [张三] [设置] [退出] │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────┐  ┌──────────────────────────────┐         │
│  │          │  │                              │         │
│  │ 侧边导航 │  │          主内容区            │         │
│  │          │  │                              │         │
│  │          │  │                              │         │
│  └──────────┘  └──────────────────────────────┘         │
└──────────────────────────────────────────────────────────┘
```

7.2 侧边导航栏

功能模块：

```
┌──────────────┐
│ 🔍 智能问答   │ ← 默认页
│ 📁 文档管理   │
│ 📊 使用统计   │
│ 👥 用户管理   │ (仅管理员可见)
│ ⚙️ 系统设置   │ (仅管理员可见)
└──────────────┘
```

会话历史（智能问答子模块）：

```
┌──────────────┐
│ 💬 历史会话   │
│ ──────────────│
│ 📌 今天       │
│  · 火灾应急流程 │ ← 可重命名
│  · 安全生产制度 │
│ ──────────────│
│ 📅 昨天       │
│  · 考勤制度查询 │
│ ──────────────│
│ 📆 本周       │
│  · ...        │
│ ──────────────│
│ [+ 新建会话]  │
└──────────────
```

八、前端开发要求

1. 技术栈

- **框架**：Next.js 14 (App Router, TypeScript)
- **UI**：Shadcn/UI, Tailwind CSS, Lucide Icons

- **编辑器**：Monaco Editor (配置为 Light Theme)

2. UI/UX 风格规范

- **色调**：浅色 Apple 风格 (Frosty Glass)
- **背景色**：`#f0f0f0` (浅灰)
- **强调色**：`#3b82f6` (蓝色)
- **字体**：`-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif`
- **布局**：顶部固定，半透明
- **导航**：顶部固定，半透明

八、智能问答项目结构

建议按照以下结构进行初始化

```
omni-knowledge/
├── backend/
│   ├── app.py
│   ├── config.py
│   ├── requirements.txt
│   └── env.example
├── api
├── frontend/
│   ├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   └── globals.css
├── README.md
├── start-backend.sh
├── start-frontend.sh
├── start.sh
└── ...
```